

计算机系统结构实验报告Lab06:

MIPS Pipelining CPU

uu-matter543

2024-2025-2

摘要

在 Lab06 中，我完成了 MIPS 32 指令流水处理器的各部件连接，实现了支持 16 条指令的指令流水功能并进行了验证；在指令流水基础上添加 Forwarding 机制以解决 Read-after-Write 数据冒险，添加 Stall 机制以解决 Load-Use 数据冒险，加入 Predict-not-taken 机制以使控制冒险的处理尽可能高效，并依次对各种机制和各项策略进行了验证；最后，在已实现模块的基础上扩展功能，使其成为能够处理 MIPS 32 指令集全部 31 条指令的指令流水 CPU，并使指令流水的性能尽可能高效，收获颇丰。

目录

1 实验目的	4
2 基本指令流水功能的实现与验证	4
2.1 指令流水基本原理	4
2.2 已有模块复用	5
2.3 流水线寄存器的设计与功能集成	6
2.3.1 IF/ID 流水线寄存器	6
2.3.2 ID/EX 流水线寄存器	7
2.3.3 EX/MEM流水线寄存器	9
2.3.4 MEM/WB 流水线寄存器	10
2.4 TOP 模块的设计与内部数据通路的连接	11
2.4.1 PC 更新逻辑	11
2.4.2 其他数据选择功能	12
2.5 基本指令流水功能的验证	12
2.5.1 仿真准备	12
2.5.2 仿真结果展示	13

3 Forwarding 机制的实现与验证	15
3.1 原理分析	15
3.2 功能实现	15
3.2.1 Forward 模块添加与 Forwarding 检测	15
3.2.2 寄存器使用情况传递	16
3.2.3 ALU 输入重定向	17
3.2.4 寄存器写操作时序逻辑调整	17
3.3 Forwarding 机制的验证	18
3.3.1 仿真准备	18
3.3.2 仿真结果展示	19
4 Stall 机制的实现与验证	19
4.1 原理分析	19
4.2 功能实现	20
4.2.1 Hazard Detection Unit 的添加与 Stall 的检测	20
4.2.2 IF/ID 流水线寄存器的处理	20
4.2.3 ID/EX 流水线寄存器的处理	21
4.2.4 PC 更新逻辑的完善	22
4.3 Stall 机制的验证	23
4.3.1 仿真准备	23
4.3.2 仿真结果展示	23
5 Predict-not-taken 机制的实现与验证	24
5.1 原理分析	24
5.2 功能实现	24
5.2.1 IF 阶段处理 j 和 jal	24
5.2.2 ID 阶段处理 beq, bne, jr	25
5.2.3 PC 更新逻辑整合	26
5.2.4 Forwarding 功能扩充	26
5.3 Predict-not-taken 策略验证	28
5.3.1 仿真准备	28
5.3.2 仿真结果展示	28
6 完整 31 条指令的支持与验证	28
6.1 I 型指令的完全支持	28
6.1.1 addiu 的支持	28
6.1.2 xori, slti, sltiu, lui 的支持	29

6.2	R型指令的完全支持	30
6.3	31 条指令的验证	31
6.3.1	仿真准备	31
6.3.2	仿真结果展示	34
7	收获与反思	36

1 实验目的

1. 理解 CPU 指令流水的基本原理、流水线冒险 (hazard) 及数据依赖关系, 在 MIPS 32 单周期 CPU 的基础上, 设计简单流水线 CPU;
2. 在以上基础上, 设计支持 Forwarding 机制的流水线 CPU, 通过检测竞争, 在冒险发生时进行数据前递, 以解决 Read-after-Write 数据冒险, 并减少因 Read-after-Write 数据冒险带来的流水线停顿延时, 提高流水线处理器性能;
3. 在以上基础上, 增加 Stall 机制, 解决 Load-Use 数据冒险, 以避免 Forwarding 机制不能解决 Load-Use 数据冒险而导致的错误;
4. 在以上基础上, 通过 Predict-not-taken 策略解决控制冒险, 最大化减少控制竞争带来的流水线停顿延时, 进一步提高处理器性能;
5. 在以上基础上, 将 CPU 支持的指令数量从 9 条或 16 条扩充为 31 条, 使处理器功能更加丰富, 基本实现支持 MIPS 32 指令集的指令流水处理器。

2 基本指令流水功能的实现与验证

2.1 指令流水基本原理

MIPS 32 指令流水处理器, 包括与数据内存和指令内存的协作, 整个微处理器系统的基本结构如下图所示:

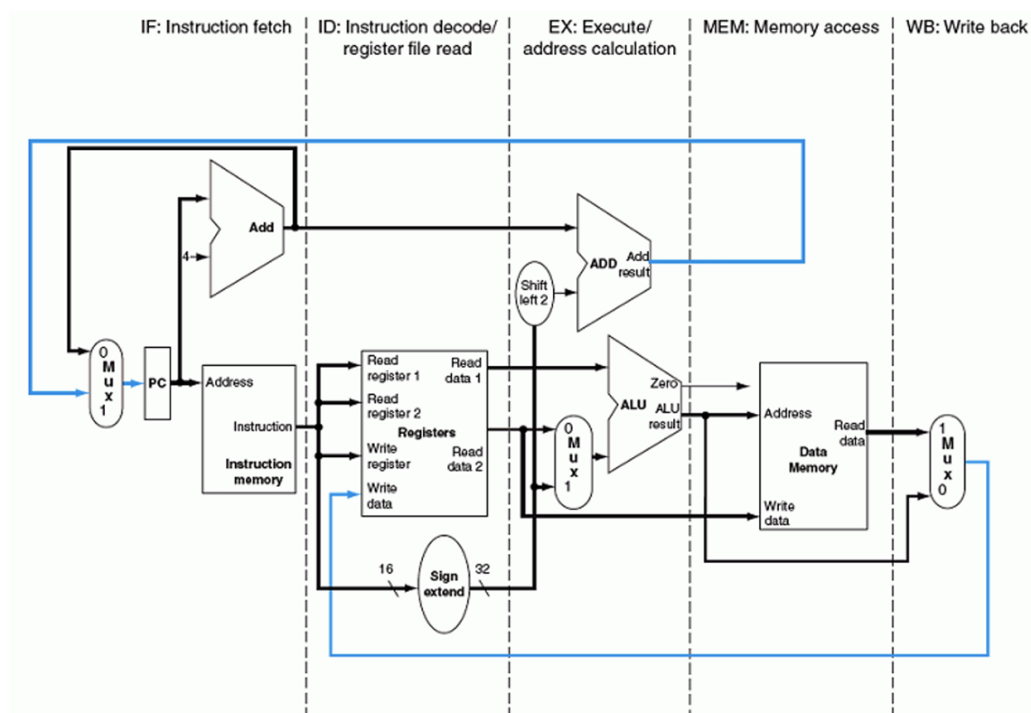


图 1: MIPS 32 指令流水处理原理图

指令流水的基本思想是：将指令的执行过程分为 5 个阶段：取指令阶段 IF、支路解码阶段 ID、ALU 运算阶段 EX、数据内存访问阶段 MEM、写回寄存器阶段 WB；没有冒险时，每条指令都在单一阶段执行，保证 CPU 各模块的独立性，保证各指令的执行不会冲突；实际应用中，指令流水可在减少单周期任务、缩短时钟周期的同时，提高单周期完成的指令数，提高处理器的吞吐量，从而提升处理器性能。

2.2 已有模块复用

Lab05 最终实现了支持 16 条指令的单周期 CPU，将其中的指令内存、控制单元、寄存器组、符号扩展模块、零扩展模块、ALU 控制单元、算术逻辑单元、数据内存直接复制到 Lab06 的工程文件中即可，因为这些部件的基本功能不变，只需在指令流水功能扩展的过程中按需做细节上的调整，如下图所示：

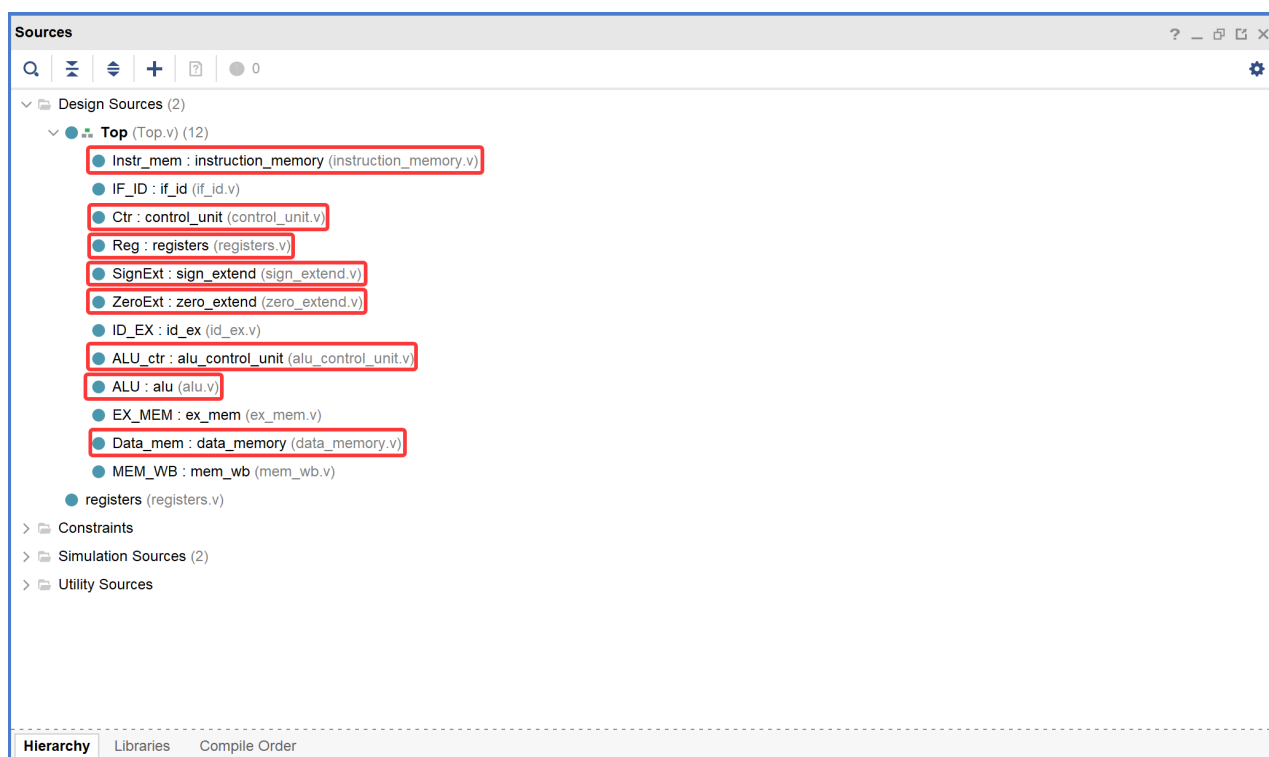


图 2: Lab05 已有模块复用

至于 PC，由于完整 31 条指令流水的实现中 PC 更新关系较为复杂，因此将 PC 控制逻辑直接嵌入 Top 模块，以减少 Top 模块数据通路的数量，同时也便于调试并修正 PC 更新关系中的错误。在基本的指令流水过程中，PC 更新关系比较简单，保证每个周期 +4 的正常更新即可。

2.3 流水线寄存器的设计与功能集成

按照实验指导书中给出的指令流水架构，创建 IF/ID, ID/EX, EX/MEM, MEM/WB 共 4 个指令流水寄存器组，以在指令流水过程中传递数据。建立各个模块与指令执行阶段之间的联系，可参照图中所示的设计文件顺序，其恰好反映了流水线寄存器按照指令执行阶段对各个模块的分组，即：指令内存位于 IF 阶段，控制单元、寄存器组（读取功能）、符号扩展模块和零扩展模块位于 ID 阶段，ALU 控制单元和算术逻辑单元位于 EX 阶段，数据内存位于 MEM 阶段，寄存器组（写入功能）位于 WB 阶段。

流水线寄存器设计的整体思路为：系统时钟下降沿，各个模块向流水线寄存器写入数据；系统时钟上升沿，流水线寄存器向各个模块发出数据。同时流水线寄存器还集成了部分功能，如数据选择，将在后续内容中介绍。

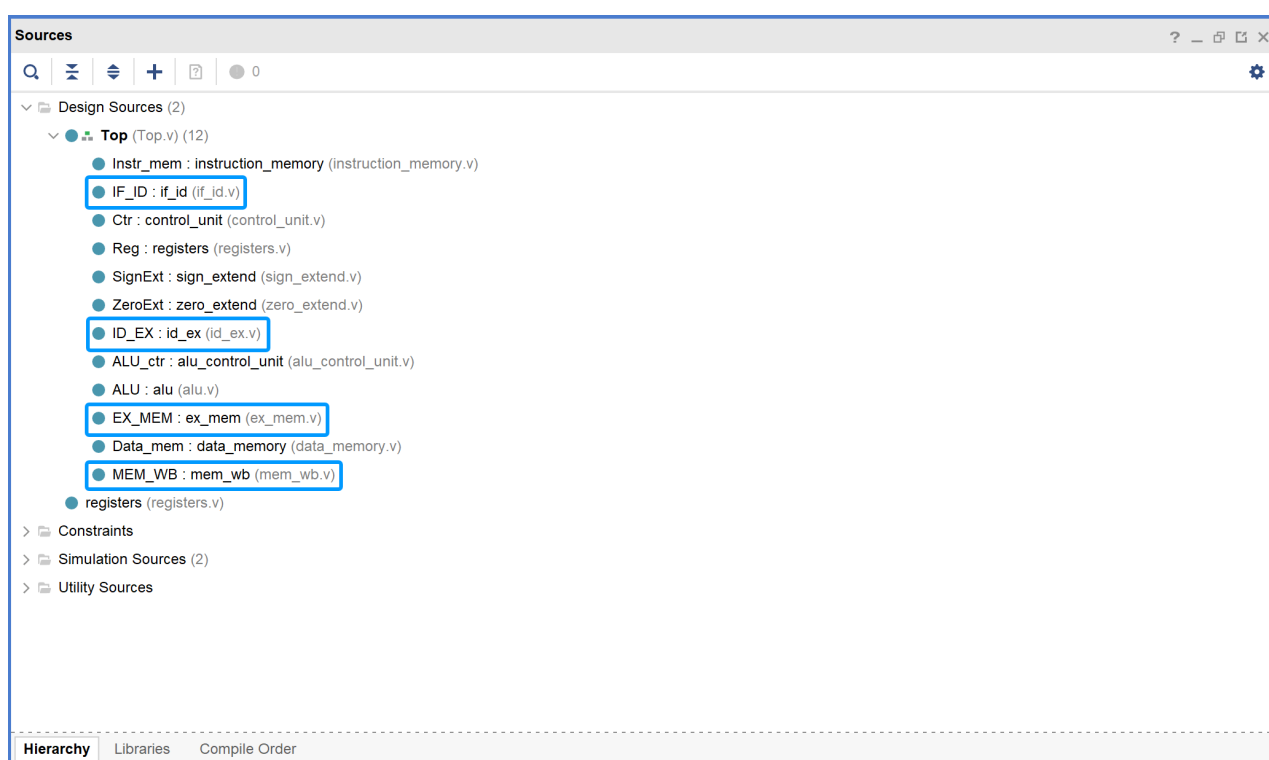


图 3: 流水线寄存器的添加

2.3.1 IF/ID 流水线寄存器

IF/ID 流水线寄存器的实现比较简单，只需要传递指令内容和 PC+4 的值即可。指令内容决定了各个模块的任务，而 PC+4 的值在 beq, bne, j, jal 等指令中作为操作数（虽然基本指令流水未涉及，但在完整 31 条指令中将会使用），所以需要向后传递。而流水线寄存器写入写出与时钟周期的协作关系定义为，在内部为每个信号设置缓冲区，同时 2 个 always 语句块分别实现：时钟上升沿将输入数据写入内部缓冲区；时钟下降沿将内部缓冲

区写出至输出寄存器。由于后续流水线寄存器含有大量信号，因此后续流水线寄存器涉及的信号中，如不特殊说明，均为上述传递机制。

```
module if_id(  
    input clock,  
    input [31:0] if_instr,  
    output reg [31:0] id_instr,  
    input [31:0] if_pc,  
    output reg [31:0] id_pc  
);  
reg [31:0] instr_buffer;  
reg [31:0] pc_buffer;  
initial begin  
    instr_buffer = 0;  
    pc_buffer = 0;  
    id_pc = 0;  
    id_instr = 0;  
end  
always @ (negedge clock) begin  
    instr_buffer <= if_instr;  
    pc_buffer <= if_pc;  
end  
always @ (posedge clock) begin  
    id_pc <= pc_buffer;  
    id_instr <= instr_buffer;  
end  
endmodule
```

2.3.2 ID/EX 流水线寄存器

在 ID 阶段，指令被解码，[31:26] 位 opcode 输入控制单元并产生控制信号；[25:21] 和 [20:16] 位输入寄存器组并产生 2 个寄存器数据；[20:16] 和 [15:11] 位作为寄存器写操作下标的可能值，直接向后传递；[15:0] 位输入符号扩展模块和零扩展模块进行扩展产生 2 个扩展 32 位数；[10:6] 位作为立即数位移运算的位数，直接向后传递；[5:0] 位决定 R 型指令的运算类型，直接向后传递。

在 ID 过程中 PC+4 的值没有用到（事实上，基本指令流水中 PC+4 只会在 PC 正常递增时用到，暂时可以忽略其他情况），直接向后传递。

输入输出端口定义如下：

```
module id_ex(  
    input clock,  
    input id_sign_ext,  
    input id_reg_dst,  
  
    input id_alu_src,  
    output reg ex_alu_src,  
    input id_mem_to_reg,  
    output reg ex_mem_to_reg,  
    input id_reg_write,  
    output reg ex_reg_write,  
    input id_mem_read,  
    output reg ex_mem_read,  
    input id_mem_write,  
    output reg ex_mem_write,  
    input id_branch,  
    output reg ex_branch,  
    input [2:0] id_alu_op,  
    output reg [2:0] ex_alu_op,  
    input [10:0] id_shamt_funct,  
    output reg [10:0] ex_shamt_funct,  
    input id_jump_load,  
    output reg ex_jump_load,  
    input [31:0] id_pc,  
    output reg [31:0] ex_pc,  
    input [31:0] reg_data_1,  
    output reg [31:0] alu_src_1,  
    input [31:0] reg_data_2,  
    output reg [31:0] alu_src_2,  
    input [31:0] after_sign_ext,  
    input [31:0] after_zero_ext,  
    output reg [31:0] after_ext,  
    input [9:0] reg_dst_opt,  
    output reg [4:0] reg_dst  
);
```

ID/EX 的数据传递中，按照默认方式传递的信号有：

- alu_src (选择 ALU 的第 2 个操作数为寄存器值或立即数)
- mem_to_reg (选择寄存器写操作的数据源是内存数据或 ALU 运算结果)

- reg_write (寄存器写使能信号)
- mem_read (数据内存读使能信号)
- mem_write (数据内存写使能信号)
- branch (条件跳转指令标志信号)
- alu_op (输入到 ALU 控制单元中决定 ALU 运算)
- shamt_funct (指令低11位)
 - shamt 为立即数位移运算的运算数
 - funct 输入到ALU控制单元中决定ALU运算
- jump_load (跳转并保存指令 jal 标志信号)
- pc (PC+4 的值)
- reg_data_1 (第 1 个寄存器数据)
- reg_data_2 (第 2 个寄存器数据)

这些信号将在 EX、MEM、WB 阶段被使用，信号传递的具体实现不再赘述。

经选择的数据有：

- after_sign_ext (符号扩展结果) 和 after_zero_ext (零扩展结果) 经 sign_ext 选择后输出到after_ext，作为 ALU 第 2 个操作数的可能值
- reg_dst (写操作目标寄存器号) 由 jump_load 和 reg_dst 共同决定：jump_load 为 1 时 reg_dst 为 \$31 即 \$ra 寄存器；否则若 reg_dst 为 1 则选择指令 [15:11] 位 (R 型指令)，为 0 则选择指令 [20:16] 位 (I 型指令)。

选择逻辑可用如下代码描述：

```
after_ext <= id_sign_ext ? signext_buffer : zeroext_buffer;
reg_dst <= id_jump_load ? 5'b11111 :
          (id_reg_dst ? reg_dst_opt[4:0]:
            reg_dst_opt[9:5]);
```

2.3.3 EX/MEM流水线寄存器

在 EX 过程中，ALU 控制单元获取 alu_op 和 funct，产生 alu_ctr_out 送入 ALU，ALU 根据 2 个运算数和 alu_ctr_out 指定的运算，产生运算结果。

输入输出端口定义如下：

```
module ex_mem(
    input clock,

    input ex_mem_to_reg,
    output reg mem_mem_to_reg,
    input ex_reg_write,
```

```
output reg mem_reg_write ,
input  ex_mem_read ,
output reg mem_mem_read ,
input  ex_mem_write ,
output reg mem_mem_write ,
input  [4:0] ex_reg_dst ,
output reg [4:0] mem_reg_dst ,

input  [31:0] ex_cont ,
output reg [31:0] mem_cont ,
input  [31:0] alu_rst ,
output reg [31:0] alu_output
);
```

EX/MEM 的数据传递中，按照默认方式传递的信号有：

- mem_to_reg (选择寄存器写操作的数据源是内存数据或 ALU 运算结果)
- reg_write (寄存器写使能信号)
- mem_read (数据内存读使能信号)
- mem_write (数据内存写使能信号)
- reg_dst (寄存器写操作下标)
- cont (sw 指令向数据内存写入的数据，即从第 2 个寄存器 \$rt 中读取的数据)
- alu_rst (ALU 的运算结果，作为写回寄存器的数据，或数据内存访问的地址)

这些信号将在 MEM、WB 阶段使用，信号传递的具体实现不再赘述。本流水线寄存器未集成数据选择功能，因此比较简单。

2.3.4 MEM/WB 流水线寄存器

在 MEM 过程中，数据内存根据输入地址，送出数据或写入数据。

输入输出端口定义如下：

```
module mem_wb(
    input  clock ,

    input  mem_reg_write ,
    output reg wb_reg_write ,
    input  [4:0] mem_reg_dst ,
    output reg [4:0] wb_reg_dst ,

    input  mem_mem_to_reg ,
```

```
input [31:0] alu_output ,
input [31:0] mem_data ,
output reg [31:0] wb_data
);
```

MEM/WB 的数据传递中，按照默认方式传递的信号有：

- reg_write (寄存器写使能信号)
- reg_dst (寄存器写操作下标)

这些信号将在 WB 阶段使用，信号传递的具体实现同样不再赘述。

经选择的数据有：alu_output (ALU 的运算结果) 和 mem_data (数据内存送出的数据) 经 mem_to_reg 选择后输出到 wb_data，作为寄存器写操作的数据源。

选择逻辑可用如下代码描述：

```
wb_data <= mem_mem_to_reg ? mem_buffer : alu_buffer;
```

至此，流水线寄存器的设置已完成。

2.4 TOP 模块的设计与内部数据通路的连接

由于指令流水机制的加入，内部数据通路数量急剧膨胀，因此需要制定适用于指令流水的命名规则。本实验中指定的命名规则为：[phase]_[signal]，其中 [phase] 表示执行阶段，[signal] 表示信号名称。位于 TOP 模块内部的数据通路，[phase] 用大写字母表示；位于子模块中的数据通路，[phase] 用小写字母表示。通过这种方式，内部数据通路的混乱程度有所降低，也使后续附加模块的设计与连接，以及调试过程有所简化。

2.4.1 PC 更新逻辑

由于后续参与运算的都是 PC+4，因此我直接将 PC+4 的值传递给 IF/ID 寄存器：

```
wire [31:0] ID_instr;
wire [31:0] ID_pc;
wire [31:0] PC_add_4;
assign PC_add_4 = PC + 4;
if_id IF_ID(
    .clock(Clk),
    .if_instr(IF_instr),
    .id_instr(ID_instr),
    .if_pc(PC_add_4),
    .id_pc(ID_pc)
);
```

基本指令流水只需要用 PC_add_4 的值，在每个时钟上升沿更新 PC 即可：

```
always@(posedge Clk) PC <= Reset ? 0 :  
    (ID_jump ? {ID_pc[31:28], ID_instr[25:0]<<2}:  
    (EX_branch && EX_zero ? (EX_alu_src_2 << 2) + EX_pc:  
    PC_add_4));
```

关于 beq, bne, j, jal, jr 等指令, 由于它们会导致控制冒险, 在基本指令流水中暂时不考虑, 测试用指令集也不会包含此类指令。

2.4.2 其他数据选择功能

在 EX 阶段, alu_src 在 ext (位扩展后的 32 位立即数) 和 alu_src_2 中, 选择 ALU 的第 2 个操作数 (这里尝试过将该数据选择关系集成至 ID/EX 流水线寄存器内部, 但由于 sw 指令仍然需要 alu_src_2 作为数据内存写操作的数据源, 还是要将 alu_src_2 向后传递, 于是采用了在 TOP 模块中选择数据的实现方式):

```
assign EX_alu_src_opt = EX_alu_src ? EX_ext : EX_alu_src_2;
```

在 EX 阶段, jump_load 对后续数据传递进行筛选, 若为 jal 指令, 则直接将流水传递的 PC+4 传递到 alu_output, 否则传递 ALU 的运算结果:

```
assign result = EX_jump_load ? EX_pc : EX_alu_result;
```

至此, 指令流水 CPU 框架的搭建已完成, 该 CPU 应能处理不含数据冒险和控制冒险的汇编代码。

2.5 基本指令流水功能的验证

2.5.1 仿真准备

数据内存初始值: 前 4 个 word 为 0, 1, 2, 3, 其余 word 均为 0, 便于验证功能。

基本指令流水功能验证使用的汇编代码如下:

```
lw $1, ($0)  
lw $2, 4($0)  
lw $3, 8($0)  
lw $4, 12($0)  
addi $5, $1, 1  
addi $6, $2, -2  
ori $7, $3, 3  
andi $8, $4, 15  
add $9, $1, $2  
sub $10, $3, $4  
and $11, $3, $7
```

```
or $12, $1, $3
slt $13, $6, $9
sll $14, $4, 2
srl $15, $4, 1
sw $8, 16($0)
sw $9, 20($0)
sw $10, 24($0)
sw $11, 28($0)
```

汇编为机器语言如下：

```
10001100000000010000000000000000
10001100000000010000000000000100
100011000000000110000000000001000
10001100000001000000000000001100
00100000001001010000000000000001
0010000001000110111111111111110
00110100011001110000000000000011
00110000100010000000000000001111
00000000001000100100100000100000
00000000011001000101000000100010
00000000011001110101100000100100
00000000001000110110000000100101
00000000110010010110100000101010
00000000000000001011100001000000
000000000000000010111100001000010
101011000000010000000000000010000
101011000000010010000000000010100
101011000000010100000000000011000
101011000000010110000000000011100
```

仿真激励文件可直接复用 Lab05 的 instr_16_tb.v 即可，不再赘述。

2.5.2 仿真结果展示

第 1 ~ 8 条指令和第 9 ~ 19 条指令的执行情况分别如图4和图5所示，可以在上述波形图中观察到明显的指令流水逻辑，说明指令流水基本框架已成功搭建。

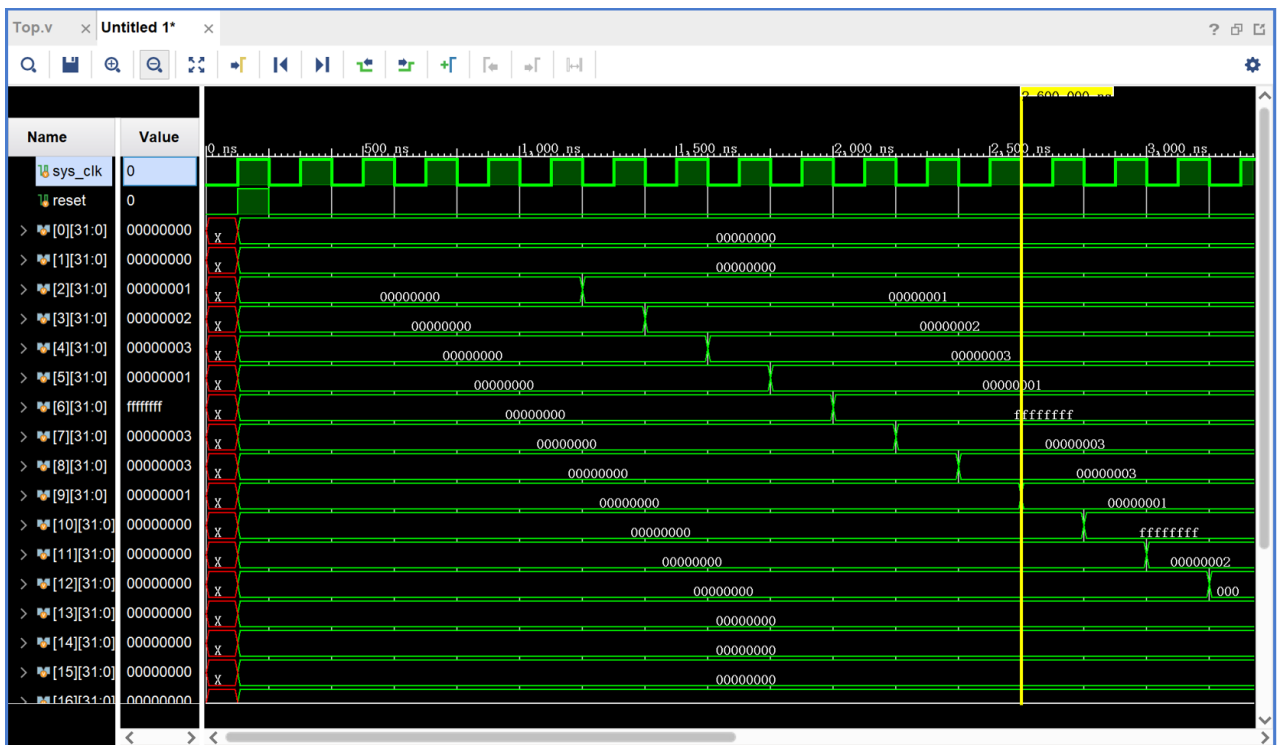


图 4: 基本指令流水逻辑的验证 1

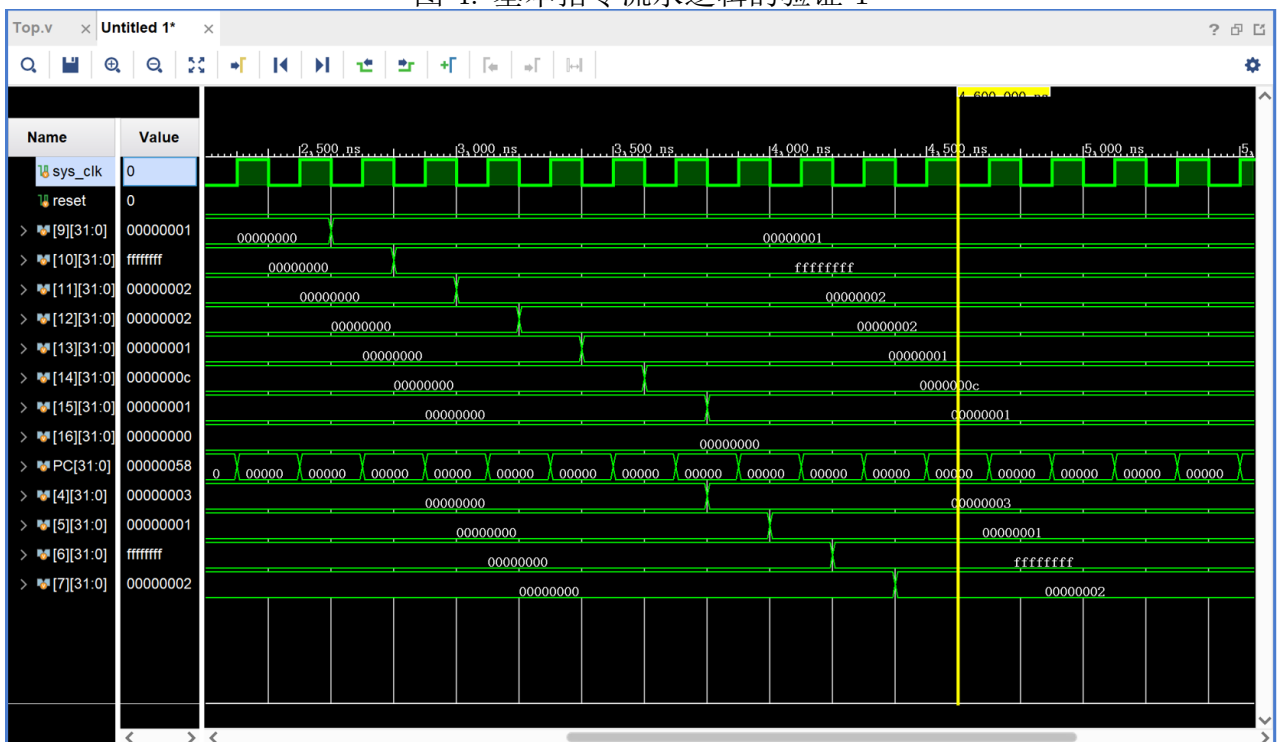


图 5: 基本指令流水逻辑的验证 2

3 Forwarding 机制的实现与验证

3.1 原理分析

我将 Forwarding 模块置于 EX 阶段，并将 ID 阶段使用的寄存器传递至 EX 阶段。当检测到 EX 阶段使用的寄存器与 MEM 或 WB 阶段的写回寄存器相同时，就将数据前递，用于解决 Read-After-Write 数据冒险。

3.2 功能实现

3.2.1 Forward 模块添加与 Forwarding 检测

新建设计文件 forward.v，用于 Forwarding 机制的检测。输入端口有：

- EX 阶段使用的 2 个寄存器
- MEM 和 WB 阶段的写使能信号和写回寄存器下标

输出端口定义为 2 位，因为需要表示 3 种情况：

- 不前递
- MEM → EX 前递
- WB → EX 前递

输入输出端口定义如下：

```
module forwarding(  
    input [4:0] ex_rs,  
    input [4:0] ex_rt,  
    input [4:0] mem_reg_dst,  
    input mem_reg_write,  
    input [4:0] wb_reg_dst,  
    input wb_reg_write,  
    output reg [1:0] forwardA,  
    output reg [1:0] forwardB  
);
```

数据前递是否发生的判断逻辑为：

1. forwardA 表示 \$rs 寄存器前递，forwardB 表示 \$rt 寄存器前递
2. forwardA 或 forwardB 值为 10 表示 MEM → EX 前递，为 01 表示 WB → EX 前递
3. 默认不前递；当寄存器写信号为 1，写回寄存器不为 0 (按照 MIPS 32 架构中寄存器的使用标准，\$zero 寄存器始终为 0，不需要前递)，写回寄存器与正在使用寄存器的值相同时，按照上述赋值标准对 forwardA 或 forwardB 进行赋值
4. 一般情况下，有可能在 MEM 和 WB 阶段的指令写回了相同的寄存器，按照指令流水的时序逻辑，MEM 阶段的数据为最新寄存器数据，应向 EX 阶段前递 MEM 阶段的

数据，因此首先判断 MEM $\rightarrow$ EX 前递是否发生，并在判断 WB $\rightarrow$ EX 前递是否发生时排除 MEM $\rightarrow$ EX 前递发生的情况，保证指令流水正确进行

按照以上逻辑编写的代码如下：

```
always @(*) begin
    // Default not to forward
    forwardA = 2'b00;
    forwardB = 2'b00;
    // EX hazard
    if (mem_reg_write && (mem_reg_dst != 0) && (mem_reg_dst == ex_rs)
        ) begin
        forwardA = 2'b10;
    end
    if (mem_reg_write && (mem_reg_dst != 0) && (mem_reg_dst == ex_rt)
        ) begin
        forwardB = 2'b10;
    end
    // MEM hazard
    if (wb_reg_write && (wb_reg_dst != 0) &&
        !(mem_reg_write && (mem_reg_dst != 0) && (mem_reg_dst == ex_rs
            )) &&
        (wb_reg_dst == ex_rs)) begin
        forwardA = 2'b01;
    end
    if (wb_reg_write && (wb_reg_dst != 0) &&
        !(mem_reg_write && (mem_reg_dst != 0) && (mem_reg_dst == ex_rt
            )) &&
        (wb_reg_dst == ex_rt)) begin
        forwardB = 2'b01;
    end
end
end
```

3.2.2 寄存器使用情况传递

由于 Forwarding 模块在 EX 部分，因此需要将 ID 阶段使用的寄存器传递到 EX 阶段进行判断。在 ID/EX 流水线寄存器中添加：

```
input [4:0] id_rs,
input [4:0] id_rt,
output reg [4:0] ex_rs,
```



```
output reg [4:0] ex_rt,
```

rs 传递指令 [25:21] 位表示的寄存器, rt 传递指令 [20:16] 位表示的寄存器。

然而这里依然存在逻辑问题。I 型指令并没有用到指令的 [20:16] 位寄存器表示的值, 而数据前递判断中却并未考虑上述 rt 未使用的情况。我的解决方式是, 因为 alu_src 在 EX 阶段选择寄存器值或立即数作为 ALU 的第 2 个操作数, 需要从 ID 阶段向 EX 阶段传递, 所以在 ID/EX 传递 rt 时, 若 alu_src 为 0 (表示选择立即数), 则 rt 强制传递 0, 从而在 Forwarding 判断时被条件“写回寄存器非 0”过滤掉, 不会导致错误的数据前递。综合以上分析, rs 和 rt 写出关系编写为:

```
ex_rs = rs_buffer;  
ex_rt = alu_src_buffer ? 5'b00000 : rt_buffer;
```

3.2.3 ALU 输入重定向

保持原有的无冒险时的数据选择关系不变, 在数据前递发生时, 选择将在 MEM 或 WB 阶段的指令结果, 前递至在 EX 阶段的指令的输入:

```
assign EX_alu_input1 = (forwardA == 2'b10) ? MEM_alu_output :  
                        (forwardA == 2'b01) ? WB_data :  
                                                EX_alu_src_1;  
assign EX_alu_input2 = (forwardB == 2'b10) ? MEM_alu_output :  
                        (forwardB == 2'b01) ? WB_data :  
                                                EX_alu_src_opt;
```

并修改ALU的输入端口:

```
alu ALU(  
    .reg_1(EX_alu_input1),  
    .reg_2(EX_alu_input2),  
    .shamt(EX_shamt_funct[10:6]),  
    .alu_ctr(EX_alu_ctr),  
    .alu_result(EX_alu_result),  
    .zero(EX_zero)  
);
```

3.2.4 寄存器写操作时序逻辑调整

Forwarding 只需要跨 1 ~ 2 条指令进行数据前递的前提条件是: 第 i 条指令的寄存器写操作在前半个阶段 (系统时钟为高电平期间) 尽快完成, 这样第 i+3 条指令在系统时钟下降沿进行寄存器读操作就能够读取到符合汇编代码逻辑的数据, 此时不需要额外添加跨 3 条指令的数据前递功能。

然而在 Lab05 单周期处理器的实现中，寄存器写操作只能在时钟下降沿进行，会与 ID 阶段的时钟下降沿从寄存器组读取数据并写入流水线寄存器产生冲突，因此需要对该部分进行加速，保证在 WB 阶段的时钟下降沿之前准备好数据。

首先，对 MEM/WB 流水线寄存器做出如下调整：时钟下降沿收到 MEM 阶段的数据后，立即发出，无需等待时钟上升沿。

```
always @ (negedge clock) begin
    reg_write_buffer = mem_reg_write;
    reg_dst_buffer = mem_reg_dst;
    mem_to_reg_buffer = mem_mem_to_reg;
    alu_buffer = alu_output;
    mem_buffer = mem_data;
    wb_reg_write = reg_write_buffer;
    wb_reg_dst = reg_dst_buffer;
    wb_data = mem_mem_to_reg ? mem_buffer : alu_buffer;
end
```

其次，对寄存器组做出如下调整：在时钟上升沿完成写入数据，这样在时钟下降沿根据指令中的寄存器下标读取寄存器值时，能够读取到符合时序逻辑的数据。

```
always @ (posedge clock)
    if (wr_en) reg_cont[wr_reg] = wr_data;
```

至此，加入 Forwarding 机制的指令流水 CPU 的搭建已完成，该 CPU 应能处理仅含 Read-After-Write 数据冒险，不含 Load-Use 数据冒险和控制冒险的汇编代码。

3.3 Forwarding 机制的验证

3.3.1 仿真准备

输入激励依然使用 Lab05 中编写的 instr_16_tb.v 即可，不再赘述；数据内存存在本次仿真中没有用到，因此简单将数据内存设为全 0 即可。

Forwarding 机制验证使用的汇编代码如下：

```
addi $1, $0, 5
addi $2, $0, 3
sub $3, $1, $2
and $4, $2, $3
or $5, $2, $3
add $6, $3, $3
add $6, $3, $6
add $6, $3, $6
```

```
add $6, $3, $6
add $6, $3, $6
add $6, $3, $6
add $6, $3, $6
```

汇编为机器语言如下：

```
00100000000000010000000000000101
00100000000000010000000000000011
00000000001000100001100000100010
00000000010000110010000000100100
00000000010000110010100000100101
00000000011000110011000000100000
00000000011001100011000000100000
00000000011001100011000000100000
00000000011001100011000000100000
00000000011001100011000000100000
00000000011001100011000000100000
00000000011001100011000000100000
00000000011001100011000000100000
```

3.3.2 仿真结果展示

可以在图6中观察到指令流水 CPU 正确处理了 Read-After-Write 数据冒险。

4 Stall 机制的实现与验证

4.1 原理分析

Forwarding 机制处理 Read-After-Write 数据冒险的原理是：第 i 条指令在 EX 阶段产生数据，根据需要送至第 $i+1$ 条指令的 EX 阶段 (MEM $\rightarrow$ EX前递) 或者第 $i+2$ 条指令的 EX 阶段 (WB $\rightarrow$ EX前递)，其能够成功处理数据冒险依赖于第 i 条指令能在 EX 阶段产生结果。而在 lw 指令的执行过程中，MEM 阶段才能产生写回寄存器的数据，因此必须在 Load 和 Use 之间加入 1 条什么也不做的空指令 (机器语言 32 位 0) 以缓冲数据依赖情况。

然而，Forwarding 机制在处理 Load-Use 数据冒险也并非毫无作用，因为 lw 指令在 MEM 阶段就能产生写回寄存器的数据，而数据到达寄存器却要在 WB 阶段，此时 Forwarding 机制便可实现数据前递。

综合以上分析，需要检查在相邻 2 个阶段执行指令的 Load-Use 关系，而且为了减少对流水效率的影响，应位于指令执行的尽早阶段。由于 ID 阶段才对指令解码，因此我将其置于 ID 和 EX 之间。检测到 Load-Use 数据冒险时，强制插入 1 条什么也不做的空指令。

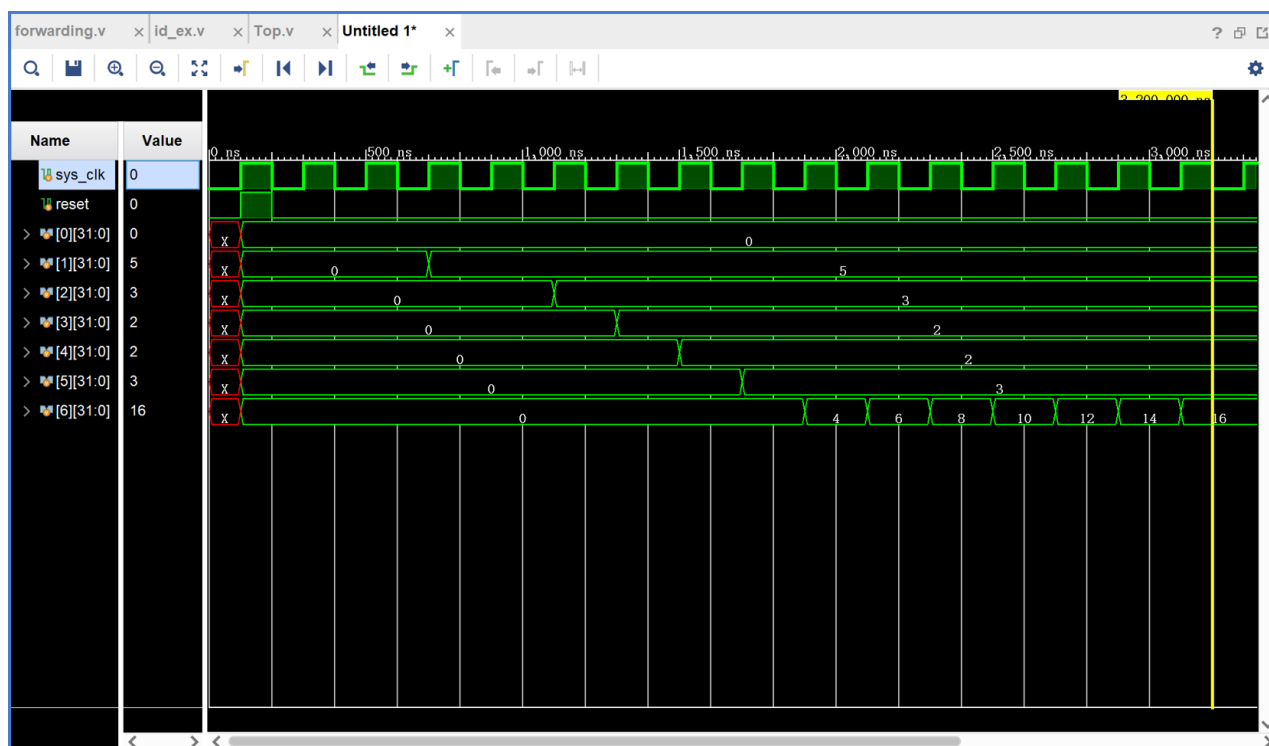


图 6: Forwarding 机制的验证

4.2 功能实现

4.2.1 Hazard Detection Unit 的添加与 Stall 的检测

新建设计文件 `hazard_detection.v`, 用于 Stall 机制的检测。冒险检测单元的设计比较简单, 只需要在 `mem_rd` (数据内存读使能信号) 为 1 时, 检测 ID 阶段使用的 2 个寄存器和 EX 阶段的 `rt` 寄存器是否相同即可。当然, 这里的 `ex_rt` 也不能为 0, 因为 `$zero` 寄存器始终为 0, 不允许写入, 因此 `ex_rt` 若为 0, 则一定不是寄存器写操作的下标。

```
always @(*) begin
    if (ex_mem_read && ((ex_rt == id_rs) || (ex_rt == id_rt)) && (
        ex_rt != 0))
        stall = 1'b1;
    else
        stall = 1'b0;
end
```

4.2.2 IF/ID 流水线寄存器的处理

由于 IF 阶段并未解码指令, 其发出的数据只要不经 ID 阶段解码, 就不会影响机器状态。因此 Stall 发生时, 将 IF/ID 流水线寄存器冻结即可, 冻结方式为不更新内部的信号缓

冲区，在相邻 2 个时钟下降沿发出同样的信号。

```
always @ (negedge clock) begin
    if (stall) begin
        instr_buffer <= instr_buffer;
        pc_buffer <= pc_buffer;
    end else begin
        instr_buffer <= if_instr;
        pc_buffer <= if_pc;
    end
end
end
```

4.2.3 ID/EX 流水线寄存器的处理

由于 IF/ID 采用上述冻结机制，ID 的各个模块会在 2 个时钟周期内收到 2 组相同的信号，因此 Stall 发生时要将 ID/EX 流水线寄存器清空，避免 Use 指令错误执行。这样 ID 阶段在 Stall 发生的相邻 2 个时钟周期，会陆续向后发送 1 组全 0 的信号表示什么也不做，发送 1 组 Use 指令的信号表示指令正常执行。

```
always @ (negedge clock) begin
    if (stall) begin
        alu_src_buffer = 0;
        mem_to_reg_buffer = 0;
        reg_write_buffer = 0;
        mem_read_buffer = 0;
        mem_write_buffer = 0;
        branch_buffer = 0;
        alu_op_buffer = 0;
        shamt_funct_buffer = 0;
        jump_load_buffer = 0;
        pc_buffer = 0;
        reg_data_1_buffer = 0;
        reg_data_2_buffer = 0;
        signext_buffer = 0;
        zeroext_buffer = 0;
        reg_dst_index_buffer = 0;
        rs_buffer = 0;
        rt_buffer = 0;
    end
    else begin
```

```

    alu_src_buffer = id_alu_src;
    mem_to_reg_buffer = id_mem_to_reg;
    reg_write_buffer = id_reg_write;
    mem_read_buffer = id_mem_read;
    mem_write_buffer = id_mem_write;
    branch_buffer = id_branch;
    alu_op_buffer = id_alu_op;
    shamt_funct_buffer = id_shamt_funct;
    jump_load_buffer = id_jump_load;
    pc_buffer = id_pc;
    reg_data_1_buffer = reg_data_1;
    reg_data_2_buffer = reg_data_2;
    signext_buffer = after_sign_ext;
    zeroext_buffer = after_zero_ext;
    reg_dst_index_buffer = reg_dst_opt;
    rs_buffer = id_rs;
    rt_buffer = id_rt;

end
end

```

4.2.4 PC 更新逻辑的完善

Stall 发生时，用 PC 值而非 PC+4 值更新 PC 值，避免 Use 指令的下条指令被错误地跳过。而关于 beq, bne, j, jal, jr 等导致控制冒险的指令，在 Stall 机制中仍暂时不做考虑。

```

always @ (posedge Clk) begin
    if (Reset)
        PC <= 0;
    else if (Stall)
        PC <= PC;
    else if (ID_jump)
        PC <= {ID_pc[31:28], ID_instr[25:0] << 2};
    else if (EX_branch && EX_zero)
        PC <= (EX_alu_src_2 << 2) + EX_pc;
    else
        PC <= PC_add_4;
end

```

至此，加入 Forwarding 机制和 Stall 机制的指令流水 CPU 的搭建已完成，该 CPU 应能处理可能含 Read-After-Write 冒险和 Load-Use 冒险，不含控制冒险的汇编代码。

4.3 Stall 机制的验证

4.3.1 仿真准备

输入激励依然使用 Lab05 中编写的 `instr_16_tb.v` 即可，不再赘述。

数据内存初始值：第 1 个 word 设为 16 进制 10，其余 word 均为 0，因为要验证 Load-Use 数据冒险的处理，需要从数据内存中读取数据，因此数据内存设置了有效内容。

Stall 机制验证使用的汇编代码如下：

```
addi $1, $0, 1
lw $2, ($0)
addi $3, $2, 1
addi $4, $3, 1
```

汇编为机器语言如下：

```
001000000000000010000000000000001
10001100000000010000000000000000
001000000100001100000000000000001
001000000110010000000000000000001
```

4.3.2 仿真结果展示

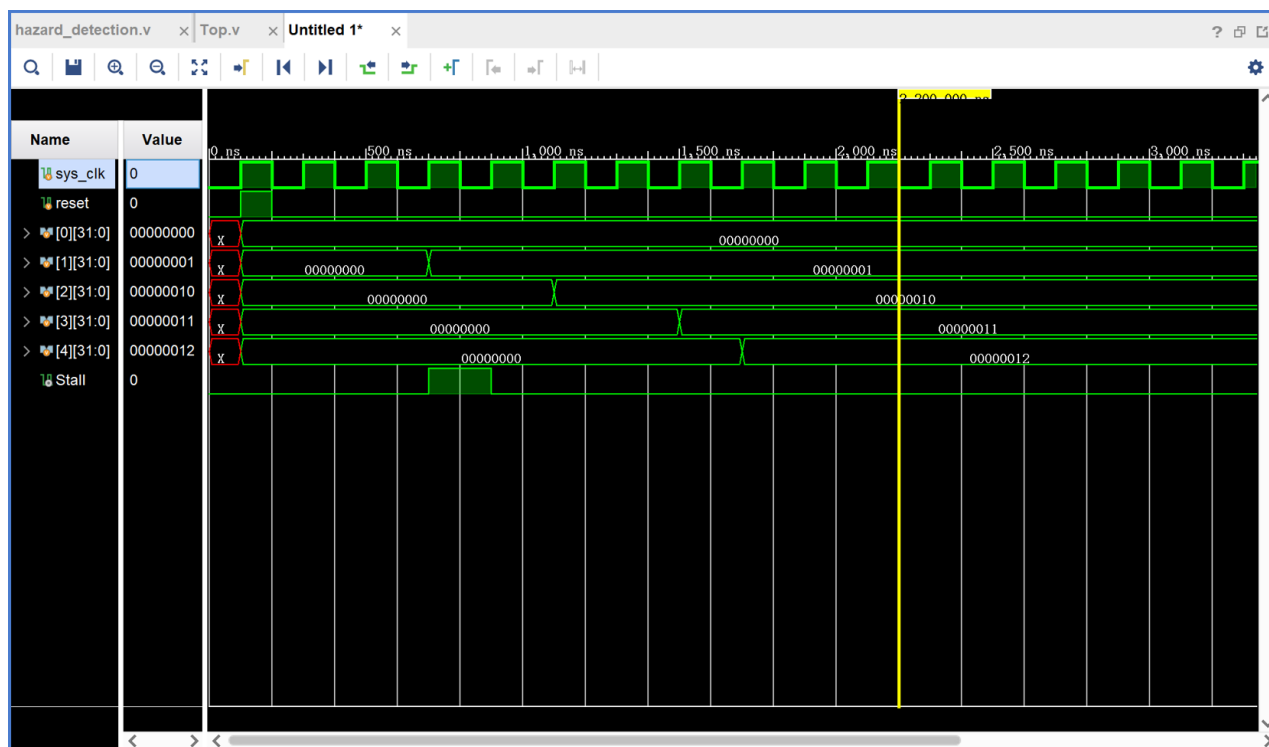


图 7: Stall 机制的验证

可以在图7中观察到 Stall 机制发挥了作用，指令流水 CPU 正确处理了 Load-Use 数据冒险，保证了 \$3 寄存器中结果的正确性。

5 Predict-not-taken 机制的实现与验证

5.1 原理分析

Forwarding 机制在不影响流水性能的基础上处理了 Read-After-Write 数据冒险，Stall 机制以 1 个 Stall 的代价处理了 Load-Use 数据冒险。剩下的问题就是如何处理控制冒险，以达到性能损失最小化？

导致控制冒险的指令有 5 条：beq, bne, j, jr, jal，根据是否需要寄存器值可分为 2 类：

- beq, bne, jr 需要读取寄存器的值，用于决定是否跳转或获取跳转的目标地址
- j, jal 则为无条件跳转，无需寄存器读操作

因此，为了达到性能最大化，j 和 jal 最早可在 IF 阶段直接处理，而 beq, bne, jr 最早可在 ID 阶段获取跳转信息。

而 beq, bne 与 jr 也有着不同之处：beq 和 bne 指令有跳转和不跳转 2 种情况，jr 则一定跳转。这 3 条指令本身最早可在 ID 阶段获取跳转信息，并且跳转信息会影响下条指令的 IF 阶段，这是导致数据冒险的根本原因。最容易想到的解决方案就是检测到这 3 条指令后，加入 1 个 Stall，从而保证下条指令的正确性，但这影响了 beq 和 bne 不跳转时的性能，因此引入了预测的解决方案。

常用的预测机制有：Predict-taken (预测跳转)，Predict-not-taken (预测不跳转)，LRU (根据最近 n 次控制冒险的跳转情况，选择跳转或不跳转) 等。由于 Predict-taken 和 Predict-not-taken 的命中率似乎相近（实际上，根据对具体程序的分析与统计，大多数分支指令的最终结果并不会跳转，进一步说明了 Predict-not-taken 策略比 Predict-taken 更具高效性），而 PC 具有在每个周期 +4 的正常更新机制，因此 Predict-not-taken 比 Predict-taken 的实现方法更为简单；同时，LRU 实现方式虽然较前 2 者而言，具有更高的预测成功率，而且正确率会随着 n 的增大而增大，但是实现起来比 Predict-taken 和 Predict-not-taken 策略更为复杂。由于实验时间有限，因此采用性价比最高的 Predict-not-taken 策略。

具体实现方式为，beq, bne, jr 指令到达 ID 阶段后，IF 阶段正常流水，但如果 ID 阶段发生了跳转，就把位于 IF 阶段的指令清除。

5.2 功能实现

5.2.1 IF 阶段处理 j 和 jal

查表得知 j 和 jal 指令的 opcode 分别为 000010 和 000011，为代码简便，可以只取 [31:27] 位，若为 00001 则跳转。根据原理分析，IF 阶段处理 j 和 jal 指令，不影响下条指

令 IF 阶段的正确性，因此可将 j 和 jal 视为 PC 的“正常更新”，无需调整流水线寄存器，直接集成到 Top 模块的 PC 更新逻辑中。

```
assign next_PC = Reset ? 0 :
(IF_instr[31:27]==5'b00001 ? {PC_add_4[31:28], IF_instr[25:0], 2'b00} :
PC_add_4);
```

5.2.2 ID 阶段处理 beq, bne, jr

条件跳转指令 beq 和 bne 的跳转判断为：解码指令的 opcode，为 000100 表示 beq 指令，为 000101 表示 bne 指令；branch_taken 表示跳转发生情况，为 1 表示跳转，为 0 表示不跳转，默认为 0；为 beq 指令时，从寄存器读取的 2 个数据相等则 branch_taken 为 1；为 bne 指令时，从寄存器读取的 2 个数据不相等则 branch_taken 为 1。

```
wire branch_taken;
assign branch_taken = ((ID_instr[31:26] == 6'b000100) & (
ID_reg_data_1_fwd == ID_reg_data_2_fwd)) | ((ID_instr[31:26] == 6'
b000101) & (ID_reg_data_1_fwd != ID_reg_data_2_fwd));
```

beq 和 bne 的跳转地址 branch_targ 用 PC+4+offset 计算，offset 即为指令 [15:0] 位表示的 16 位立即数经符号扩展并左移 2 位后的结果。

```
wire [31:0] branch_targ;
assign branch_targ = ID_pc + (ID_signext << 2);
```

无条件跳转到寄存器值指令 jr 的判定为：将指令的 [5:0] 位即 funct 一并输入控制单元，当且仅当 opcode 为 000000 (R 型指令) 且 funct 为 001000 (jr 指令) 时 jr 信号为 1，表示跳转到寄存器值。

```
case (op_code)
6'b000000: begin // R
    reg_dst = 1;
    alu_src = 0;
    mem_to_reg = 0;
    reg_write = 1;
    mem_read = 0;
    mem_write = 0;
    branch = 0;
    alu_op = 3'b100;
    jump = 0;
    if (funct == 6'b001000) jr = 1;
    sign_ext = 0;
```

```

    jump_load = 0;
end

```

jr 的跳转地址 jr_targ 即为寄存器组的第 1 个读取数据。

```

wire [31:0] jr_targ;
assign jr_targ = ID_reg_data_1;

```

最后合并 beq, bne, jr 的跳转地址到 renew_pc 信号处, 并在 2 类跳转发生时, 设置 Flush 信号值为 1, 作为清除错误指令的信号。

```

wire [31:0] renew_pc;
assign renew_pc = ID_jr ? jr_targ :
    (branch_taken ? branch_targ :
        PC_add_4);
assign Flush = branch_taken || ID_jr;

```

5.2.3 PC 更新逻辑整合

当 Reset 为 1 时表示重置 CPU, PC 设为 0; 当 Stall 为 1 时表示插入空指令, PC 不变; 当 Flush 为 1 时表示发生了 beq, bne, j 跳转, PC 设为 renew_pc; 其余情况即为 PC 的“正常更新”, 包括 PC+4 和 j, jal 导致的跳转。

```

always @(posedge Clk) begin
if (Reset)
    PC <= 0;
else if (Stall)
    PC <= PC;
else if (Flush)
    PC <= renew_pc;
else
    PC <= next_PC;
end

```

5.2.4 Forwarding 功能扩充

由于将 beq, bne, jr 指令的主要执行部分提前到了 ID 阶段, 因此前述 MEM→EX 和 WB→EX 数据前递功能难以达到目的, 因此仿照已实现的数据前递, 扩充了 EX→ID, MEM→ID, WB→ID 的数据前递功能。

输出端口添加 forward_id_rs 和 forward_id_rt, 分别表示向 rs 和 rt 的数据前递。

```

output reg [1:0] forward_id_rs,
output reg [1:0] forward_id_rt

```

forward_id_rs 和 forward_id_rt 默认为 00，为 01 表示 WB → ID 数据前递，为 10 表示 MEM → ID 数据前递，为 11 表示 EX → ID 数据前递，在原有向 EX 数据前递的机制保持不变的基础上，添加如下向 ID 数据传递的判断（此处也考虑了多重数据冒险，即 ID 和 EX, MEM, WB 中多个待执行的寄存器写操作都有冲突时，优先传递距 ID 阶段更近的寄存器写操作将写回的数据）：

```
forward_id_rs = 2'b00;
if (id_rs != 0) begin
    if (ex_reg_write && (ex_reg_dst == id_rs))
        forward_id_rs = 2'b11; // EX->ID
    else if (mem_reg_write && (mem_reg_dst == id_rs))
        forward_id_rs = 2'b10; // MEM->ID
    else if (wb_reg_write && (wb_reg_dst == id_rs))
        forward_id_rs = 2'b01; // WB->ID
end
forward_id_rt = 2'b00;
if (id_rt != 0) begin
    if (ex_reg_write && (ex_reg_dst == id_rt))
        forward_id_rt = 2'b11; // EX->ID
    else if (mem_reg_write && (mem_reg_dst == id_rt))
        forward_id_rt = 2'b10; // MEM->ID
    else if (wb_reg_write && (wb_reg_dst == id_rt))
        forward_id_rt = 2'b01; // WB->ID
end
```

并为 ID 阶段使用的从寄存器读取的数据添加选择关系：

```
assign ID_reg_data_1_fwd = (Forward_id_rs==2'b11) ? EX_alu_result :
                           (Forward_id_rs==2'b10) ? MEM_alu_output :
                           (Forward_id_rs==2'b01) ? WB_data :
                           ID_reg_data_1;
assign ID_reg_data_2_fwd = (Forward_id_rt==2'b11) ? EX_alu_result :
                           (Forward_id_rt==2'b10) ? MEM_alu_output :
                           (Forward_id_rt==2'b01) ? WB_data :
                           ID_reg_data_2;
```

ID_reg_data_1_fwd 和 ID_reg_data_2_fwd 只在 beq, bne, jr 的判断中使用，不影响前述向 EX 数据前递的机制，因此对 ALU 等模块的输入不做修改，继续使用已实现的机制。

至此，加入 Forwarding 机制、Stall 机制、Predict-not-taken 策略的指令流水 CPU 的搭建已完成，该 CPU 应能处理含 Read-After-Write 数据冒险、Load-Use 数

据冒险和控制冒险的汇编代码。

5.3 Predict-not-taken 策略验证

5.3.1 仿真准备

输入激励依然使用 Lab05 中编写的 `instr_16_tb.v` 即可，不再赘述；数据内存在本次仿真中没有用到，因此简单将数据内存设为全 0 即可。

Predict-not-taken 策略验证使用的汇编代码如下：

```
addi $1, $0, 3
addi $2, $0, 3
beq $1, $2, label
addi $3, $0, 1
label:
addi $4, $0, 1
```

汇编为机器语言如下：

```
00100000000000010000000000000011
00100000000000010000000000000011
00010000001000100000000000000001
00100000000000110000000000000001
00100000000000100000000000000001
```

5.3.2 仿真结果展示

可在图8中看到，Predict-not-taken 策略成功激活 Flush 信号，将 `addi $3, $0, 1` 指令清除，避免了错误，保证了 \$3 结果的正确性。

6 完整 31 条指令的支持与验证

6.1 I 型指令的完全支持

6.1.1 addiu 的支持

`addiu` 指令的处理可基于 `addi` 指令，比较简单。鉴于有符号数和无符号数的运算只影响对计算机存储的数据的解释，而并不影响数据在计算机内部的存储形式，这里选择将 2 条指令一并处理。在控制单元内部，`addi` 和 `addiu` 的 opcode 前 5 位都是 00100，只需要根据最低位设置符号扩展或零扩展即可；而由于 `addi` 和 `addiu` 实质上都是加法操作，ALU 控制单元和 ALU 均无需修改。

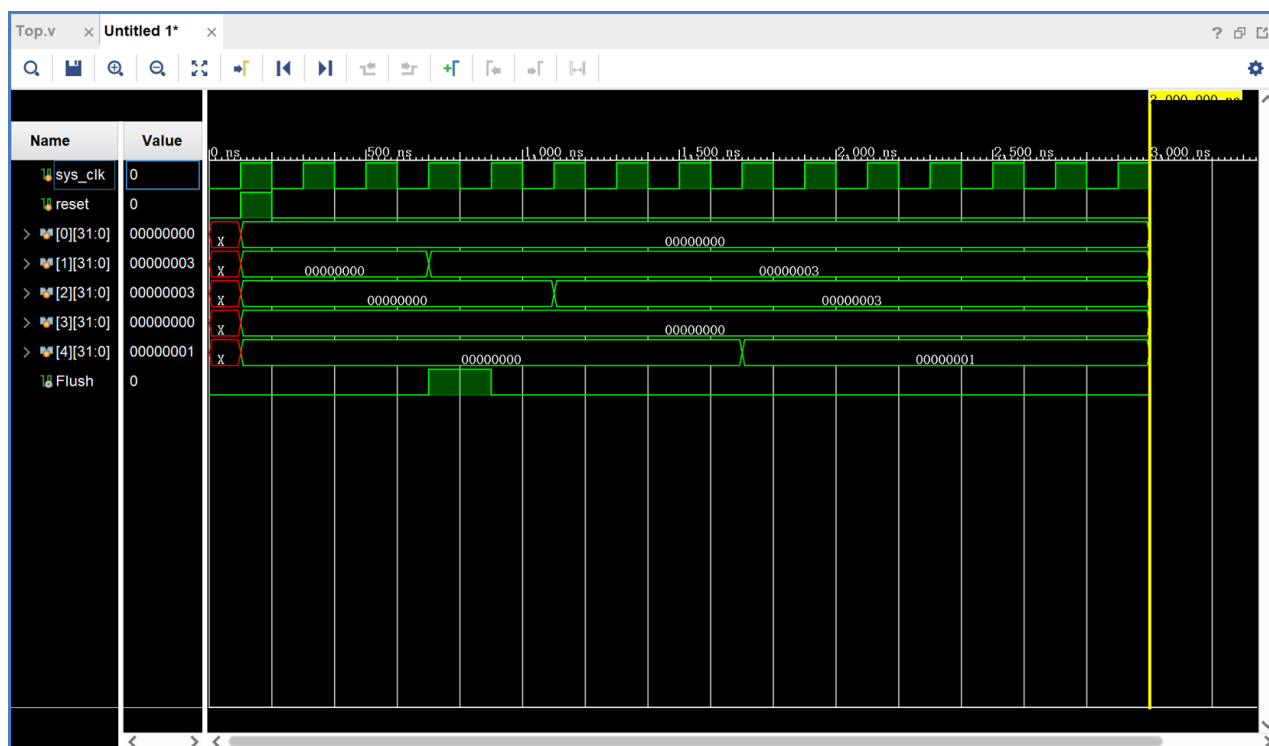


图 8: Predict-not-taken 策略的验证

```

6'b00100x: begin // addi, addiu
    reg_dst = 0;
    alu_src = 1;
    mem_to_reg = 0;
    reg_write = 1;
    mem_read = 0;
    mem_write = 0;
    branch = 0;
    alu_op = 3'b000;
    jump = 0;
    jr = 0;
    if (op_code[0] == 0) sign_ext = 1;
    else sign_ext = 0;
    jump_load = 0;
end

```

6.1.2 xori, slti, sltiu, lui 的支持

这几条指令的执行框架、控制信号与 addi 和 addiu 相似，但也有不同之处，即 ALU 需要进行不同操作，因此只需要适当调整 alu_op 即可，其他控制信号均与 addi 和 addiu 相

同。alu_op 与 ALU 中运算的关系如下：

```
alu_op = 3'b000; //addi, addiu, lw, sw
alu_op = 3'b001; //lui
alu_op = 3'b010; //andi
alu_op = 3'b011; //ori
alu_op = 3'b100; //xori
alu_op = 3'b101; //slti
alu_op = 3'b110; //sltiu
```

alu_op 为 3 位，最多有 8 种比特串，在 I 型指令使用 7 种后，恰好剩余 alu_op 为 111 的情况，供 R 型指令使用，因此不需要扩展 alu_op 的位数。

对于 I 型指令，指令的 [5:0] 位是立即数的低 6 位，即 funct 与 ALU 中的运算无关，ALU 控制单元需要根据 alu_op 直接决定 ALU 中进行的操作，关系如下：

```
//I-type
9'b000xxxxxx: alu_ctr_out <= 4'b0010; //add
9'b001xxxxxx: alu_ctr_out <= 4'b0011; //lui
9'b010xxxxxx: alu_ctr_out <= 4'b0000; //and
9'b011xxxxxx: alu_ctr_out <= 4'b0001; //or
9'b100xxxxxx: alu_ctr_out <= 4'b1101; //xor
9'b101xxxxxx: alu_ctr_out <= 4'b0111; //slt
9'b110xxxxxx: alu_ctr_out <= 4'b0101; //sltu
```

ALU 根据 alu_ctr_out 进行运算的部分，将在考虑 R 型指令后进行说明。

6.2 R型指令的完全支持

R 型指令则由于有统一的指令格式，添加功能更为简单，只需要统筹好 alu_ctr_out 和 ALU 中运算的关系即可。R 型指令对应的 ALU 控制单元输出如下：

```
9'b11110000x: alu_ctr_out <= 4'b0010; //add, addu
9'b11110001x: alu_ctr_out <= 4'b0110; //sub, subu
9'b111100100: alu_ctr_out <= 4'b0000; //and
9'b111100101: alu_ctr_out <= 4'b0001; //or
9'b111100110: alu_ctr_out <= 4'b1101; //xor
9'b111100111: alu_ctr_out <= 4'b1100; //nor
9'b111101010: alu_ctr_out <= 4'b0111; //slt
9'b111101011: alu_ctr_out <= 4'b0101; //sltu
9'b111000000: alu_ctr_out <= 4'b1110; //sll
9'b111000010: alu_ctr_out <= 4'b1111; //srl
9'b111000011: alu_ctr_out <= 4'b1010; //sra
```

```

9'b111000100: alu_ctr_out <= 4'b1001; //sllv
9'b111000110: alu_ctr_out <= 4'b1011; //srlv
9'b111000111: alu_ctr_out <= 4'b1000; //srav
9'b111001000: alu_ctr_out <= 4'b0100; //jr

```

alu_ctr_out 为 4 位，最多有 16 种比特串，R 型指令的运算集与 I 型指令的运算集的并集的规模恰好为 16，因此也不需要扩展 alu_ctr_out 的位数，注意 alu_ctr_out 与 ALU 中运算的对应关系即可。

ALU 中运算与 alu_ctr_out 的关系如下：

```

4'b0000: alu_result = reg_1 & reg_2; //and
4'b0001: alu_result = reg_1 | reg_2; //or
4'b0010: alu_result = reg_1 + reg_2; //add
4'b0011: alu_result = reg_1 - reg_2; //sub
4'b0100: alu_result = reg_2 << 16; //lui
4'b0101: alu_result = $signed(reg_1) < $signed(reg_2) ? 1 : 0; //slt
4'b0110: alu_result = reg_1 < reg_2 ? 1 : 0; //sltu
4'b0111: alu_result = ~(reg_1 | reg_2); //nor
4'b1000: alu_result = reg_1 ^ reg_2; //xor
4'b1001: alu_result = reg_2 <<< shamt; //sll
4'b1010: alu_result = reg_2 >>> shamt; //srl
4'b1011: alu_result = reg_2 >> shamt; //sra
4'b1100: alu_result = 32'h00000000; //jr
4'b1101: alu_result = reg_2 <<< reg_1; //sllv
4'b1110: alu_result = reg_2 >>> reg_1; //srlv
4'b1111: alu_result = reg_2 >> reg_1; //sra

```

总体而言，Lab05 已经实现的 16 条指令，都是完整 31 条指令中有复杂数据转移关系的指令，剩下的指令执行过程都与 16 条指令中的部分 R 型指令或部分 I 型指令相同，因此将 16 条指令扩展到 31 条指令的过程十分简单。

6.3 31 条指令的验证

6.3.1 仿真准备

数据内存初始值为 0, 1, 2, 3, 4, 5, 6, 7, 8，其余 word 全为 0；使用的汇编代码如下：

```

main:
    jal label
    lui $2, 12345
    j end

```

```
label:
    lw $1, 8($0)
    lw $1, 12($0)
    lw $2, 16($0)
    add $3, $1, $2
    and $3, $1, $2
    addi $3, $0, 6
    sub $4, $1, $2
    sll $1, $1, 1
    sll $3, $3, 3
    or $1, $3, $1
    addiu $2, $0, 6
    slti $1, $1, 1
    ori $1, $2, 1
    sllv $1, $3, $2
    addu $3, $1, $2
    sub $3, $1, $2
    subu $3, $1, $2
    or $3, $1, $2
    xor $3, $1, $2
    nor $3, $1, $2
    slt $3, $1, $2
    sra $1, $1, 1
    sw $2, 4($0)
    sltu $1, $0, $2
    srl $2, $2, 2
    srav $2, $2, $2
    xori $2, $2, 2
    sltiu $1, $2, 4
    beq $5, $0, label1
    bne $5, $0, label2
label1:
    jr $ra
label2:
    xor $1, $1, $1
    addi $1, $1, 666
end:
    j end
```


汇编为机器语言如下：

```
0000110000000000000000000000000011
00111100000000100011000000111001
0000100000000000000000000000100100
1000110000000000100000000000001000
100011000000000010000000000001100
100011000000001000000000000010000
00000000001000100001100000100000
00000000001000100001100000100100
00100000000000110000000000000110
00000000001000100010000000100010
000000000000000010000100001000000
00000000000000110001100011000000
00000000011000010000100000100101
00100100000000100000000000000110
00101000001000010000000000000001
00110100010000010000000000000001
000000000010000110000100000000100
00000000001000100001100000100001
00000000001000100001100000100010
00000000001000100001100000100011
00000000001000100001100000100101
00000000001000100001100000100110
00000000001000100001100000100111
00000000001000100001100000101010
000000000000000010000100001000011
10101100000000100000000000000100
0000000000000000100000100000101011
0000000000000000100001000010000010
000000000010000100001000000000111
00111000010000100000000000000010
001011000100000100000000000000100
000100001010000000000000000000001
000101001010000000000000000000001
000000111110000000000000000001000
00000000001000010000100000100110
00100000001000010000001010011010
000010000000000000000000000100100
```



```

sub $3, $1, $2 # $3 <= 3066
subu $3, $1, $2 # $3 <= 3066
or $3, $1, $2 # $3 <= 3078
xor $3, $1, $2 # $3 <= 3078
nor $3, $1, $2 # $3 <= 3079
slt $3, $1, $2 # $3 <= 0

```

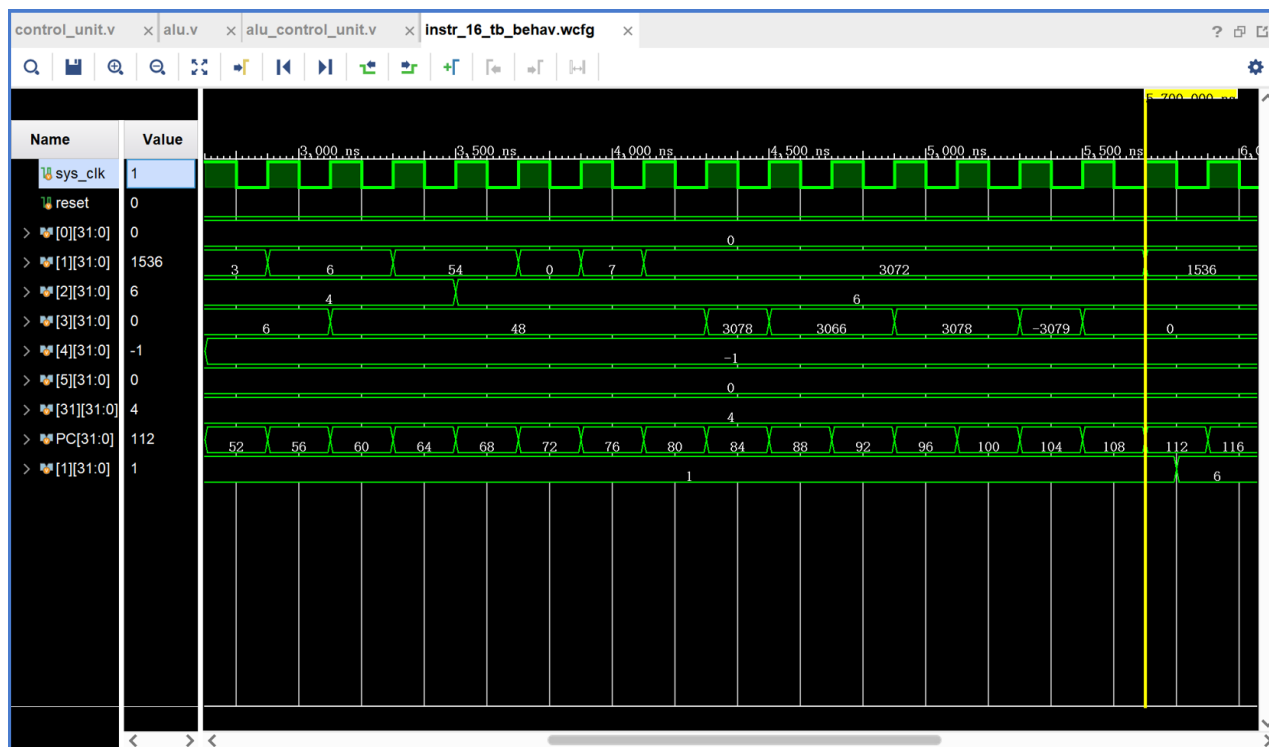


图 10: 完整 31 条指令的验证 2

```

sra $1, $1, 1 # $1 <= 1536
sw $2, 4($0) # mem[1] <= 6
sltu $1, $0, $2 # $1 <= 1
srl $2, $2, 2 # $2 <= 1
srav $2, $2, $2 # $2 <= 0
xori $2, $2, 2 # $2 <= 2
sltiu $1, $2, 4 # $1 <= 1
beq $5, $0, label1 # PC <= 132
bne $5, $0, label2 # Flushed
jr $ra # PC <= $31 (Value: 4)
xor $1, $1, $1 # Flushed
lui $2, 12345 # $2 <= 809041920 (809041920==12345*65536)
j end # PC <= 144 (and loops infinitely)

```

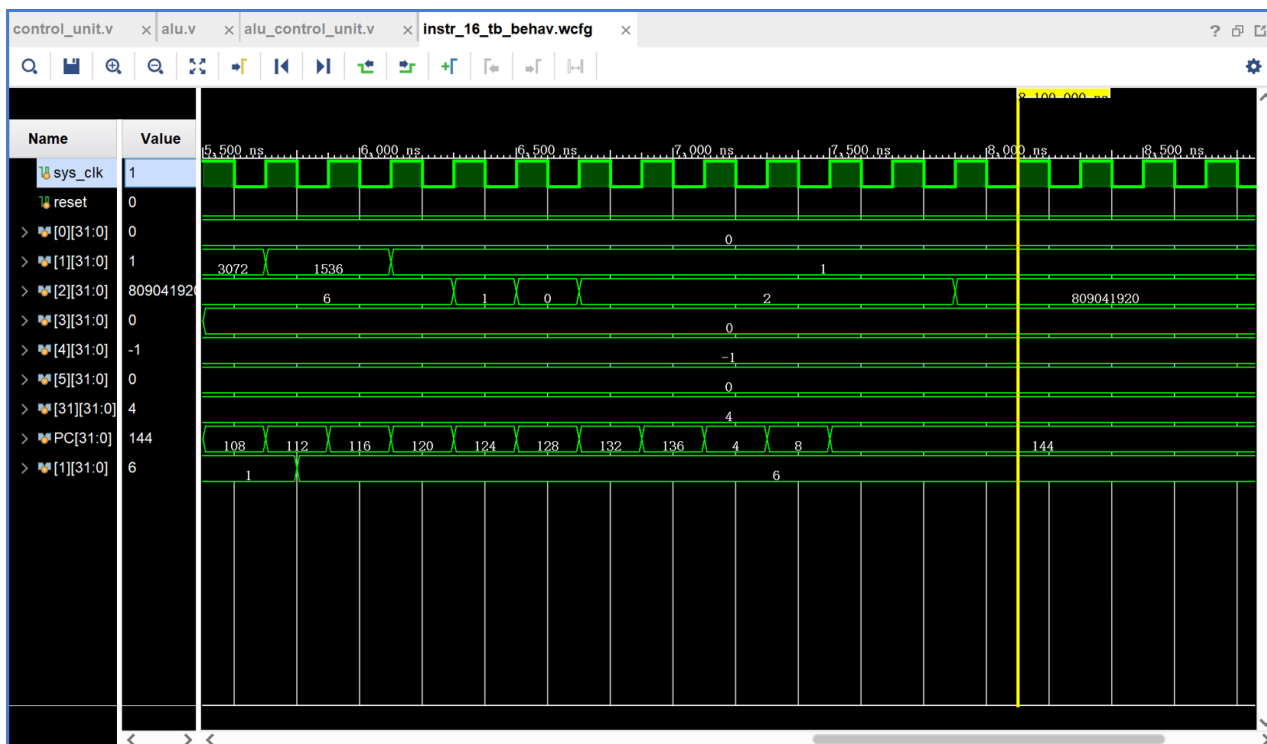


图 11: 完整 31 条指令的验证 3

全部指令执行结果核对无误，初步证明实现的简易 MIPS 32 指令流水处理器能够处理 MIPS 32 指令集的全部 31 条指令。

7 收获与反思

本次实验我由简单到复杂，实现了支持 MIPS 32 指令集全部 31 条指令的流水线处理器。在实现过程中我遇到了较多困难，比较突出的有两个：实现数据前递时，我按照计算机系统结构理论课上学到的方法，让数据前递过程变得十分复杂并且难以调试，于是借鉴 AI 大模型给出的建议，进行了思路上的重构，让数据前递执行变得相对规整；实现预测不跳转策略的过程中，为减少因预测错误导致的性能损失，我将分支指令的跳转判断提前到指令解码阶段，但是花费了较长时间才发现，分支指令本身也会造成数据冒险，而且原先实现的数据前递并不支持分支指令在指令解码阶段判断，因此我又对数据前递进行了扩充。

实验过程中我选择把流水线寄存器作为通用的模块，数据和信号都要手动定义，这种设计在最后看来各有优劣。优点是每个信号都有着自己的名称，在调试过程中能够精确探查错误的根源；缺点是需要大量的数据通路，以支持各种信号的指令流水传递。为在一定程度上避免大量数据通路带来的不利影响，我规定了一套命名规则，以便能快速找到需要观察波形变化的信号，这一点在一定程度上节省了调试花费的时间。

在实验中，我每实现一部分功能，就编写对应的汇编代码，并汇编为指令进行功能验证，保证每一部分都是正确的。在每一部分功能实现时就备份代码文件，并在复制出来的

工程文件上添加功能。这一点为我省下了大量调试的时间；同时，在数据前递的实现过程中，因为思路混乱，导致基本指令流水功能也被破坏，而因为备份，可以随时从已经验证过基本指令流水功能的代码文件上重新开始实现数据前递，减少了不必要的损失。

每一部分功能测试用的汇编代码是我手动编写的，而对应的机器码是由汇编器生成的，保证本实验实现的处理器具有一定的通用性，能和流通的指令集执行过程相匹配；最后的完整 31 条指令集测试用例由助教提供，具有一定的代表性，同时也再次验证了前面实现的附加功能没有问题。

有所不足的是，完整 31 条指令的仿真过程中，发现前 2 条从内存向第 1 个寄存器写入数据的中间有 1 个不必要的停顿，可能是因为冒险判断有问题，也可能是因为前几条指令受到重置信号的影响。但由于数据前递模块有 2 套独立的数据前递机制，比较臃肿，难以调试并发现错误，因此并未加以修复，但不影响指令流水结果的正确性。

最后，再次感谢助教和老师的悉心指导，也感谢课程组编写的全部 6 份指导书、提供的测试用例，以及对计算机系统结构实验课程的整体设计，从而让我在完成全部 6 个实验的过程中对 CPU 的架构有了更深的了解，也对 FPGA, Vaidado, Verilog 都有了一定的了解，收获颇丰。